AI-POWERED NETWORK ANOMALY DETECTION SYSTEM

A PROJECT REPORT

Submitted by

ONGKAR PAL (23SCSE1012814)

for

Web Technology Comprehensive Course

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

GALGOTIAS UNIVERSITY, GREATER NOIDA

May 2026

BONAFIDE CERTIFICATE

This is to certify that Project Report entitled "AI-Powered Network Anomaly Detection System" which is submitted by Ongkar Pal (23SCSE1012814) in partial fulfillment of the requirements for the award of the B. Tech. (Computer Science and Engineering) submitted in the School of Computing Science and Engineering of Galgotias University, Greater Noida, India is a record of the candidate's own work carried out.

Signature of Student

Date: May, 2026

Place: Greater Noida

ABSTRACT

AI-Powered Network Anomaly Detection System is a full-stack, real-time cybersecurity platform designed to monitor, detect, and respond to network-level threats using a combination of rule-based logic, machine learning, and behavioral analytics. Unlike conventional network monitoring tools that rely solely on static signature matching, this system employs multiple layers of intelligent detection including an Isolation Forest-based ML engine, a User Entity Behavior Analytics (UEBA) module, a sliding-window Rate Tracker for DDoS/Port Scan detection, and an Intrusion Prevention System (IPS) that actively blocks malicious IPs at the Windows Firewall level.

The platform captures live network traffic using the Scapy packet sniffing library, processes each packet through four independent detection engines, and generates enriched security alerts with Explainable AI (XAI) reasoning and a calculated Threat Score. Alerts are persisted in a PostgreSQL database via SQLAlchemy ORM and are served to a premium, real-time SOC (Security Operations Center) Dashboard built with HTML5, CSS3, and Vanilla JavaScript. The dashboard features a 3D interactive network topology graph, live alert feeds with risk-based filtering, Top Attacker analytics, and an AI Security Copilot chatbot powered by Google Gemini for natural language threat analysis.

Additional capabilities include global threat intelligence lookups via the AbuseIPDB API, automated PDF incident report generation using the FPDF library, and a dedicated Attack Simulator script for controlled testing. The backend API is built using FastAPI and Uvicorn, ensuring high-performance asynchronous request handling.

The main objective of this project is to demonstrate how modern AI/ML techniques and web technologies can be combined to build an enterprise-grade, intelligent network defense system. The project serves as a strong foundation for future enhancements such as deep learning-based detection, cloud deployment, and SOC team collaboration features.

TABLE OF CONTENTS

Sr. No.	Chapter Content
1	CHAPTER 1: INTRODUCTION
2	CHAPTER 2: SOFTWARE TOOLS AND HARDWARE
3	CHAPTER 3: SYSTEM ARCHITECTURE AND METHODOLOGY
4	CHAPTER 4: SAMPLE CODE AND OUTPUT
5	CHAPTER 5: CONCLUSION

CHAPTER 1: INTRODUCTION

In the modern digital landscape, cybersecurity threats are evolving at an unprecedented pace. Organizations face a growing number of sophisticated attacks including Distributed Denial of Service (DDoS) floods, port scanning, data exfiltration, and insider threats. Traditional Intrusion Detection Systems (IDS) that rely on predefined rule-based signatures are often unable to detect zero-day attacks or subtle behavioral anomalies that do not match known patterns. This creates a critical need for intelligent, adaptive detection systems that can learn from network behavior and respond proactively.

AI-Powered Network Anomaly Detection System is a full-stack cybersecurity platform developed to address this challenge. The system captures live network packets from the host machine's network interface, analyzes them through multiple detection engines in real-time, and presents actionable intelligence through a premium Security Operations Center (SOC) Dashboard. The platform goes beyond simple monitoring by integrating machine learning, behavioral profiling, global threat intelligence, and automated active defense mechanisms.

The project provides a comprehensive architecture where a Python-based backend handles packet capture, anomaly detection, and data persistence, while a modern JavaScript-based frontend provides real-time visualization of network activity, security alerts, and AI-generated incident response plans.

Key Features of the Application:

1. Real-Time Packet Capture

Uses the Scapy library to capture live TCP, UDP, and ICMP packets from the host machine's network interface, extracting metadata such as source/destination IPs, ports, protocol type, and packet size.

2. Rule-Based Anomaly Detection

Implements predefined security rules to flag suspicious activities such as oversized packets (>1500 bytes), access attempts on dangerous ports (22, 23, 445, 3389), and ICMP flood patterns.

3. Machine Learning Detection (Isolation Forest)

Trains an Isolation Forest model on historical traffic data to identify statistically anomalous packets that deviate from normal network behavior patterns.

4. User Entity Behavior Analytics (UEBA)

Profiles the baseline traffic rate for each IP address using an Exponential Moving Average and detects behavioral spikes exceeding 300% of the established baseline.

5. Rate-Based DDoS and Port Scan Detection

Implements a sliding time-window tracker that monitors packet rates per IP and triggers alerts when an IP exceeds 100 packets within a 30-second window.

6. Threat Score Calculation and Risk Assessment

Calculates a composite Threat Score (0-100) for each alert based on packet size, protocol type, and destination port, classifying alerts as LOW, MEDIUM, or HIGH risk.

7. Explainable AI (XAI) Alert Enrichment

Each alert includes human-readable explanations detailing why it was flagged, such as "Packet size unusually high (1600 bytes)" or "Destination port 445 is frequently targeted."

8. Global Threat Intelligence (AbuseIPDB)

Cross-references attacker IPs against the AbuseIPDB global threat database to determine if the IP is known to be malicious, boosting the Threat Score accordingly.

9. Intrusion Prevention System (IPS)

Automatically blocks HIGH-risk attacker IPs by creating inbound block rules in the Windows Firewall using the netsh command.

10. PostgreSQL Database with SQLAlchemy ORM

Persists all captured traffic logs and generated alerts in a PostgreSQL database using SQLAlchemy models and CRUD operations.

11. Real-Time SOC Dashboard

A premium, glassmorphism-styled dashboard built with HTML5, CSS3, and JavaScript that displays live statistics, a 3D interactive network topology graph, top attackers, threat summaries, and a filterable alert table.

12. AI-Powered Incident Response (Google Gemini)

Integrates with the Google Gemini API to generate AI-driven Incident Response Plans including Threat Analysis, Immediate Actions, and Mitigation Strategies.

13. AI Security Copilot Chatbot

A floating chatbot widget on the dashboard that allows SOC analysts to ask natural language questions about the current network threat landscape.

14. Automated PDF Report Generation

Generates downloadable PDF Incident Response Reports for individual alerts using the FPDF library, suitable for compliance and documentation purposes.

15. Attack Simulator

A dedicated Python script that uses Scapy to generate controlled simulated attacks (UDP Flood, Data Exfiltration, Malformed Packets) for testing and demonstration.

16. Asynchronous DNS Resolution

A background DNS resolution utility that translates raw IP addresses to human-readable domain names without adding latency to API responses.

17. FastAPI Backend with CORS Support

The entire backend API is built using FastAPI with Uvicorn, providing high-performance asynchronous HTTP endpoints with full CORS support.

CHAPTER 2: SOFTWARE TOOLS AND HARDWARE

SOFTWARE USED:

- Python 3.10+
- FastAPI (Backend API Framework)
- Uvicorn (ASGI Server)
- Scapy (Packet Capture and Network Analysis)
- Scikit-learn (Machine Learning - Isolation Forest)
- Pandas (Data Processing and CSV Handling)
- NumPy (Numerical Computation)
- SQLAlchemy (ORM for PostgreSQL)
- Psycopg2 (PostgreSQL Database Adapter)
- FPDF (PDF Report Generation)
- Google GenAI SDK (Gemini AI Integration)
- python-dotenv (Environment Variable Management)
- Requests (HTTP Client for Threat Intel API)
- HTML5
- CSS3 (Glassmorphism, Flexbox, Grid, Animations)
- JavaScript (Vanilla - Fetch API, DOM Manipulation)
- 3D Force Graph (3D Network Topology Visualization)
- Google Fonts (Inter, Outfit, Space Mono)
- Font Awesome (Icon Library)
- PostgreSQL (Relational Database)
- Visual Studio Code (IDE)

HARDWARE USED:

- Processor : Intel i5 / Ryzen 5 or above
- RAM : 8 GB or above
- Storage : 512 MB free space minimum
- Display : Full HD Monitor

- Network : Active Wi-Fi or Ethernet adapter (required for packet capture)

CHAPTER 3: SYSTEM ARCHITECTURE AND METHODOLOGY

Methodology:

The development of the AI-Powered Network Anomaly Detection System followed a systematic and layered approach to ensure that the platform is modular, scalable, and capable of providing real-time threat detection and response. The methodology adopted for this project consists of the following phases:

1. Requirement Analysis

The first phase involved studying existing network security tools and identifying their limitations. It was observed that most conventional IDS tools rely solely on static signature matching and lack the ability to detect unknown or zero-day threats. They also typically provide minimal context about why an alert was generated, making it difficult for analysts to prioritize and respond effectively.

Based on this analysis, the major requirements of the system were identified:

- Real-time packet capture from the network interface
- Multi-layered anomaly detection (Rule-based + ML + Behavioral + Rate-based)
- Threat Score calculation and risk classification
- Explainable AI (XAI) for alert reasoning
- Persistent storage of traffic logs and alerts
- Real-time SOC Dashboard with interactive visualizations
- AI-powered incident response generation
- Active defense (automated IP blocking)
- Attack simulation for controlled testing

2. System Design and Architecture

After gathering the requirements, the overall system architecture was designed using a modular, layered approach. The application was divided into the following major modules:

- a) Packet Capture Layer - Scapy-based sniffer that intercepts live network traffic.
- b) Detection Engine Layer - Four independent detectors: Rule-Based, ML (Isolation Forest), UEBA (behavioral profiling), Rate Tracker (volumetric analysis).
- c) Alert Processing Layer - Threat Score calculation, XAI enrichment, Threat Intel lookup, IPS trigger.
- d) Data Persistence Layer - PostgreSQL database with SQLAlchemy ORM.
- e) API Layer - FastAPI REST endpoints serving data to the frontend.

- f) Presentation Layer - Real-time SOC Dashboard with 3D graph, alerts table, chatbot.

3. Backend Development

a) Packet Capture (sniffer.py)

The Scapy library is used to capture live packets from the host machine's active network interface. Each packet with an IP layer is processed to extract metadata including timestamp, source IP, destination IP, protocol (TCP/UDP/ICMP), source port, destination port, and packet size. The extracted data is passed to all four detection engines and saved to both the PostgreSQL database and a CSV log file.

b) Rule-Based Detection (rule_based_detector.py)

This module implements static security rules for known threat patterns:

- Large Packet Attack: Packets exceeding 1500 bytes are flagged.
- Suspicious Port Access: Access to ports 22 (SSH), 23 (Telnet), 445 (SMB), and 3389 (RDP) is flagged.
- ICMP Flood: All ICMP traffic is flagged as potentially malicious.

c) Machine Learning Detection (ml_detector.py)

An Isolation Forest model from Scikit-learn is trained on historical packet data stored in CSV format. The model uses Protocol (label-encoded) and Packet Size as features. Once trained on at least 50 packets, it classifies each incoming packet as normal (1) or anomalous (-1). A contamination rate of 5% is used for sensitivity tuning.

d) UEBA Engine (ueba_engine.py)

The User Entity Behavior Analytics engine creates per-IP behavioral profiles that track the average data transfer rate (bytes/second) over 5-second sliding windows. The baseline is maintained using an Exponential Moving Average (80% history, 20% new data). If an IP's current traffic rate spikes above 300% of its baseline AND exceeds 5000 bytes/sec, a "UEBA Behavioral Spike Detected" alert is generated. A 15-second cooldown prevents alert spam.

e) Rate Tracker (rate_tracker.py)

This module implements a sliding time-window algorithm to detect volumetric attacks such as DDoS floods and port scans. It tracks the number of packets generated by each source IP within a configurable time window (default: 30 seconds). If an IP exceeds the threshold (default: 100 packets), a "High-Velocity Packet Flood" alert is triggered.

f) Alert Manager (alert_manager.py)

The central alert processing module orchestrates threat enrichment:

- Calculates a Threat Score based on packet size, protocol, and destination port.
- Generates Explainable AI (XAI) bullets describing why the alert was raised.

- Queries the AbuseIPDB API for global threat intelligence on the source IP.
- Boosts the Threat Score by 35 points if the IP is known to be malicious.
- Triggers the IPS engine to block HIGH-risk IPs at the firewall level.
- Saves the enriched alert to the PostgreSQL database.

g) Threat Intelligence (threat_intel.py)

Cross-references external IPs against the AbuseIPDB API using a REST call. Returns a confidence score, country of origin, and a maliciousness flag. Includes a mock simulation mode for demonstrations without an API key.

h) Intrusion Prevention System (ips_engine.py)

Actively blocks malicious IPs by executing Windows Firewall commands (netsh advfirewall firewall add rule) to create inbound block rules. Requires administrator privileges to execute.

i) Database Layer (connection.py, models.py, crud.py)

SQLAlchemy ORM is used to define two database models - TrafficLog and Alert - mapped to PostgreSQL tables. CRUD operations handle saving packets, saving alerts, and retrieving recent alerts with risk-based filtering and sorting.

4. API Development (FastAPI)

The FastAPI framework provides the following REST API endpoints:

Endpoint	Method	Description
/	GET	Serves the frontend dashboard (index.html)
/traffic	GET	Returns traffic statistics from CSV logs
/alerts	GET	Returns all recent alerts from the database
/stats	GET	Returns aggregate statistics (total packets, alerts)
/top-attackers	GET	Returns top 5 attacker IPs by alert count
/threat-summary	GET	Returns alert counts grouped by risk level
/recent-alerts	GET	Returns the 20 most recent alerts
/graph-data	GET	Returns nodes and links for 3D network graph
/generate-report	POST	Generates AI Incident Response Plan via Gemini
/chat	POST	AI Chatbot endpoint for threat Q&A
/export-pdf/{id}	GET	Generates and downloads a PDF incident report

5. Frontend Development - SOC Dashboard

a) HTML5 Structure

The dashboard is built as a single-page application with semantic HTML5 elements. It features a header with

live AI engine status indicator, a statistics grid, a 3D network graph panel, top attackers panel, threat summary panel with interactive filtering, a detailed alerts table, an AI report modal, and a floating chatbot widget.

b) CSS3 Design

The interface uses a premium dark glassmorphism design aesthetic with:

- CSS Custom Properties for a consistent design system
- Radial gradient backgrounds for depth
- Backdrop blur (glassmorphism) for panels
- Custom neon color palette (blue, purple, red, green, orange)
- Animated pulse indicator for live status
- Hover transitions and smooth micro-animations
- Responsive grid layouts using CSS Grid and Flexbox
- Monospace typography (Space Mono) for IP addresses and data

c) JavaScript Functionality

Vanilla JavaScript handles all dynamic behavior:

- Fetches data from all API endpoints using the Fetch API
- Renders real-time statistics with animated counter transitions
- Initializes and updates a 3D Force-Directed Graph using the 3D-Force-Graph library
- Implements risk-based alert filtering (HIGH/MEDIUM/LOW/ALL)
- Triggers AI report generation via POST request and renders formatted response in a modal
- Manages the floating chatbot widget with message history
- Auto-refreshes the dashboard every 5 seconds for real-time updates

6. Attack Simulation (simulate_attack.py)

A dedicated simulation script uses Scapy to generate controlled attack traffic:

- DDoS Simulation: Sends 110 spoofed UDP packets from a fake attacker IP to trigger the Rate Tracker threshold.
- Data Exfiltration Simulation: Sends a single oversized packet (1600 bytes) to trigger both the Rule-Based and UEBA detectors.
- Malformed Packet Simulation: Sends a TCP packet with unusual flags (FPU) on dangerous ports (23, 445) to trigger the ML detector.

7. Testing and Validation

The application was tested throughout the development process:

- Packet Capture Testing - Verified TCP, UDP, and ICMP packet extraction
- Rule-Based Detection Testing - Verified alerts for large packets, suspicious ports, and ICMP floods
- ML Model Training Testing - Verified model training with sufficient data and anomaly prediction
- UEBA Engine Testing - Verified baseline profiling and behavioral spike detection
- Rate Tracker Testing - Verified DDoS flood detection with the attack simulator
- Alert Processing Testing - Verified Threat Score calculation, XAI enrichment, and database persistence
- API Endpoint Testing - Verified all FastAPI endpoints return correct JSON responses
- Dashboard Testing - Verified real-time data rendering, filtering, and interactive elements
- AI Report Generation Testing - Verified Gemini API integration and mock fallback
- PDF Export Testing - Verified downloadable PDF report generation
- Cross-browser Compatibility Testing - Tested on Chrome, Edge, and Firefox
- Attack Simulator Testing - Verified end-to-end detection pipeline with simulated attacks

8. Future Enhancements

In future versions, the project can be enhanced by integrating:

- Deep Learning Models (LSTM, Autoencoders) for sequential anomaly detection
- Cloud Deployment on AWS/Azure/GCP
- Docker Containerization for portable deployment
- Role-Based Access Control (RBAC) for multi-analyst SOC teams
- Email/SMS Alert Notifications
- SIEM Integration (Splunk, Elastic)
- Network Flow Visualization with GeoIP mapping
- Advanced UEBA with user session tracking
- Automated Threat Hunting workflows
- Mobile-responsive dashboard optimization

CHAPTER 4: SAMPLE CODE AND OUTPUT

This chapter presents some of the important code snippets used in the development of the AI-Powered Network Anomaly Detection System.

1. Packet Capture and Processing (sniffer.py)

The packet sniffer captures live network packets and routes them through all detection engines.

```
from scapy.all import sniff, IP, TCP, UDP, ICMP
from detection.rule_based_detector import detect_anomaly
from detection.ml_detector import NetworkMLDetector
from detection.ueba_engine import UEBAEngine
from detection.rate_tracker import RateTracker

ai_detector = NetworkMLDetector(csv_file)
ueba_detector = UEBAEngine()
rate_detector = RateTracker(time_window=30, threshold=100)

def process_packet(packet):
    if packet.haslayer(IP):
        source_ip = packet[IP].src
        destination_ip = packet[IP].dst
        packet_size = len(packet)

        detect_anomaly(source_ip, destination_ip,
                       protocol_name, source_port,
                       destination_port, packet_size)
        ai_detector.predict_packet(...)
        ueba_detector.analyze_behavior(...)
        rate_detector.check_rate(...)
        save_packet(...)

sniff(prn=process_packet, store=False)
```

2. ML Anomaly Detection (ml_detector.py)

Isolation Forest model trained on historical traffic data.

```
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import LabelEncoder

class NetworkMLDetector:
    def __init__(self, csv_file_path):
        self.model = IsolationForest(
            contamination=0.05, random_state=42
```

```
        )
        self.encoder = LabelEncoder()
        self._train_model()

    def predict_packet(self, source_ip, ...):
        protocol_encoded = self.encoder.transform(
            [str(protocol)])[0]
        prediction = self.model.predict(
            [[protocol_encoded, packet_size]])[0]
        if prediction == -1:
            create_alert("AI Detected Anomaly", ...)
```

3. UEBA Behavioral Analysis (ueba_engine.py)

Profiles per-IP traffic baselines and detects behavioral spikes.

```
class UEBAEngine:
    def __init__(self):
        self.ip_profiles = {}
        self.WINDOW_SIZE = 5

    def analyze_behavior(self, source_ip, ...):
        current_rate = profile["window_bytes"] / duration
        # Anomaly: >300% of baseline AND >5000 bytes/sec
        if (current_rate > baseline * 3
                and current_rate > 5000):
            create_alert("UEBA Behavioral Spike", ...)

        # Exponential Moving Average update
        profile["baseline"] = (baseline * 0.8)
                              + (current_rate * 0.2)
```

4. Rate Tracker - DDoS Detection (rate_tracker.py)

Sliding window algorithm for high-velocity packet flood detection.

```
class RateTracker:
    def __init__(self, time_window=30, threshold=100):
        self.ip_history = defaultdict(list)

    def check_rate(self, source_ip, protocol, ...):
        self.ip_history[source_ip].append(time.time())
        cutoff = current_time - self.time_window
        self.ip_history[source_ip] = [
            t for t in self.ip_history[source_ip]
            if t > cutoff]
        if len(self.ip_history[source_ip]) > threshold:
            create_alert("High-Velocity Packet Flood",
                         severity="HIGH", ...)
```

5. Alert Manager with XAI (alert_manager.py)

Enriches alerts with explanations, threat scores, and global intelligence.

```
def create_alert(alert_type, severity, source_ip, ...):
    score, risk = calculate_threat_score(
        packet_size, protocol, destination_port)

    # Explainable AI (XAI) Formatting
    xai_bullets = [description]
    if packet_size > 1000:
        xai_bullets.append("Packet size unusually high")

    # Global Threat Intelligence Check
    intel = check_ip_reputation(source_ip)
    if intel["is_known_malicious"]:
        score = min(100.0, score + 35.0)

    # Active Defense (IPS)
    if risk_level == "HIGH":
        block_ip(source_ip)

    save_alert(alert_type, severity, ...)
```

6. IPS Engine (ips_engine.py)

Blocks attacker IPs at the Windows Firewall level.

```
def block_ip(ip_address):
    cmd = ('netsh advfirewall firewall add rule '
           'name="Sentinel_Block_{}" '
           'dir=in action=block '
           'remoteip={}').format(ip_address, ip_address)
    result = subprocess.run(cmd, shell=True,
                            capture_output=True)
    if result.returncode == 0:
        print("[IPS] SUCCESS: Blocked", ip_address)
```

7. AI Report Generation (llm_api.py)

Generates AI-powered Incident Response Plans using Google Gemini.

```
@router.post("/generate-report")
def generate_report(alert: AlertRequest):
    prompt = """You are an expert SOC Analyst.
    Write an Incident Response Plan:
    Alert Type: {}, Severity: {}
    Source IP: {}, Protocol: {}
    1. Threat Analysis
```

```
    2. Immediate Actions
    3. Mitigation Strategy"""

    client = genai.Client(api_key=api_key)
    response = client.models.generate_content(
        model='gemini-2.5-flash', contents=prompt)
    return {"report": response.text}
```

8. SOC Dashboard - Real-Time Loading (index.html)

Fetches data from all API endpoints and renders it dynamically.

```
async function loadDashboard() {
    const [stats, traffic, attackers, threat, alerts]
        = await Promise.all([
        fetch('/stats'), fetch('/traffic'),
        fetch('/top-attackers'),
        fetch('/threat-summary'), fetch('/alerts')
    ]);
    const data = await stats.json();
    animateValue("totalPackets", data.total_packets);
    animateValue("totalAlerts", data.total_alerts);

    // 3D Network Graph
    graph = ForceGraph3D()(elem)
        .nodeColor(n => n.group===1 ? '#38bdf8':'#f43f5e')
        .graphData(graphData);
}
setInterval(loadDashboard, 5000);
```

9. Attack Simulator (simulate_attack.py)

Generates controlled attack traffic for testing.

```
from scapy.all import IP, TCP, UDP, send

def simulate_ddos():
    for i in range(110):
        pkt = IP(src="185.15.59.22", dst="8.8.8.8") \
              / UDP(dport=80) / b"DDoS_PAYLOAD"
        send(pkt, verbose=False)

def simulate_exfiltration():
    payload = b"X" * 1600
    pkt = IP(src="91.240.118.68", dst="8.8.8.8") \
          / UDP(dport=443) / payload
    send(pkt, verbose=False)
```

Application Features & Output:

- Real-Time SOC Dashboard with Dark Glassmorphism UI
- Live Packet Capture Statistics (Total Packets, Total Alerts)
- 3D Interactive Network Topology Graph
- Top Attacker IP Analytics
- Threat Summary with Risk-Based Filtering (HIGH/MEDIUM/LOW)
- Detailed Security Events Table with XAI Explanations
- AI-Generated Incident Response Plans (via Google Gemini)
- Downloadable PDF Incident Reports
- AI Security Copilot Chatbot
- Console-Based Alert Logging
- Windows Firewall IP Blocking (Active Defense)

APPLICATION SCREENSHOTS:

Screenshot 1: Sentinel SOC - Main Dashboard View

[INSERT SCREENSHOT: Save your dashboard screenshot as 'dashboard_main.png' in the reports folder]

The main Sentinel SOC Dashboard running live at 127.0.0.1:8000 displays: Header with 'AI ENGINE ACTIVE' live indicator, four stat cards (Total Packets: 1,422 | Total Alerts: 1,416 | High Risk Anomalies: 10 | Avg Threat Score: 886), a 3D Active Network Topography graph with blue (internal) and red (external) nodes, Top Attackers panel listing the most active attacker IPs, Threat Summary panel with clickable HIGH (10) / MEDIUM (8,318) / LOW (1,642) risk filters, and the AI Security Copilot chatbot toggle.

Screenshot 2: Recent Security Events - Alert Table

[INSERT SCREENSHOT: Save your alerts screenshot as 'dashboard_alerts.png' in the reports folder]

The Recent Security Events table displays alerts with columns: TIME, ALERT TYPE, SOURCE IP, RISK LEVEL (color-coded badges - red for HIGH, orange for MEDIUM, green for LOW), SCORE, and ACTION buttons (AI Plan and PDF). Each alert row includes expandable XAI explanation panels showing the reasoning behind the detection, including OSINT threat intelligence data such as 'Known Malicious IP! Confidence: 85%. Origin: DE'. Alert types shown include AI Detected Anomaly, Large Packet Attack, and UEBA Behavioral Spike Detected.

CHAPTER 5: CONCLUSION

The development of the AI-Powered Network Anomaly Detection System has successfully demonstrated the concept of an intelligent, multi-layered cybersecurity platform that captures, analyzes, and responds to network-level threats in real-time. Unlike conventional Intrusion Detection Systems that rely solely on static signature matching, this system employs four independent detection engines - Rule-Based, Machine Learning (Isolation Forest), User Entity Behavior Analytics (UEBA), and Rate-Based Volumetric Analysis - providing comprehensive threat coverage across multiple attack vectors.

The project provides a premium Security Operations Center (SOC) Dashboard where analysts can monitor live network activity, view enriched security alerts with Explainable AI reasoning, generate AI-powered Incident Response Plans using Google Gemini, and download PDF reports for compliance documentation. The 3D interactive network topology graph provides an intuitive visualization of network connections and traffic patterns, while the AI Security Copilot chatbot enables natural language interaction with the system.

The application has been developed using Python (FastAPI, Scapy, Scikit-learn, SQLAlchemy) for the backend and HTML5, CSS3, and Vanilla JavaScript for the frontend, demonstrating the practical integration of modern AI/ML techniques with full-stack web development. The PostgreSQL database ensures persistent and reliable data storage, while the AbuseIPDB integration adds global threat intelligence capabilities.

Key Achievements of the Project:

- Developed a real-time packet capture and analysis pipeline using Scapy.
- Implemented four independent anomaly detection engines (Rule-Based, ML, UEBA, Rate Tracker).
- Trained an Isolation Forest ML model for unsupervised anomaly detection.
- Built a UEBA engine with Exponential Moving Average baseline profiling.
- Implemented a sliding-window Rate Tracker for DDoS/Port Scan detection.
- Developed a Threat Score calculation system with risk classification (LOW/MEDIUM/HIGH).
- Integrated Explainable AI (XAI) for human-readable alert reasoning.
- Integrated AbuseIPDB global threat intelligence for IP reputation checking.
- Implemented an active IPS that blocks malicious IPs via Windows Firewall.
- Built a premium SOC Dashboard with glassmorphism design, 3D network graph, and real-time updates.
- Integrated Google Gemini AI for Incident Response Plan generation and Security Copilot chatbot.
- Implemented automated PDF report generation for compliance documentation.
- Created an Attack Simulator for controlled end-to-end testing.

- Built a high-performance REST API using FastAPI with full CORS support.
- Designed a clean, responsive, and visually stunning user interface.

Although the current version provides a robust and fully functional detection platform, the project establishes a strong foundation for future enhancements. The system can be extended by integrating deep learning models (LSTM, Autoencoders) for sequential pattern detection, deploying to cloud infrastructure (AWS/Azure), containerizing with Docker, implementing role-based access control for SOC teams, adding email/SMS alert notifications, and integrating with enterprise SIEM platforms like Splunk or Elastic.

In conclusion, the AI-Powered Network Anomaly Detection System successfully achieves its objective of building an intelligent, proactive network defense platform. The project not only demonstrates practical skills in AI/ML, full-stack web development, and cybersecurity but also presents an innovative solution for real-time threat detection and automated incident response that can be adapted for enterprise security environments.

REFERENCES

1. Scikit-learn Documentation - Isolation Forest
https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html

2. Scapy - Interactive Packet Manipulation Library
https://scapy.readthedocs.io/

3. FastAPI - Modern Python Web Framework
https://fastapi.tiangolo.com/

4. SQLAlchemy - Python ORM
https://www.sqlalchemy.org/

5. AbuseIPDB - IP Address Threat Intelligence API
https://www.abuseipdb.com/

6. Google Gemini API Documentation
https://ai.google.dev/

7. 3D Force-Directed Graph - WebGL Visualization
https://github.com/vasturiano/3d-force-graph

8. FPDF - Python PDF Generation Library
https://pyfpdf.readthedocs.io/

9. PostgreSQL - Open Source Relational Database
https://www.postgresql.org/

10. Leaflet + OpenStreetMap - Interactive Map Library
https://leafletjs.com/